TRAKYA ÜNIVERSITESI · BILGISAYAR MÜHENDISLIĞI

Dosya Organizasyonu

Hafta 1–7 · Kodlar

Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

1

Tam kodlar

Kopyala, derle, çalıştır

PO1 · HAFTA 3 · 1 / 3

Binary file (ikili dosyaya) yazma, güncelleme ve okuma

```
#include <array>
#include <cstring>
#include <fstream>
#include <iostream>
#include <vector>
using namespace std;

struct Kayit {
    array<char, 21> ad;
    int no;
    double puan;
};

Kayit yap(const char* ad, int no, double puan) {
    Kayit k{};
    strncpy(k.ad.data(), ad, k.ad.size() - 1);
    k.no = no;
    k.puan = puan;
    return k;
}
```

PO1 · HAFTA 3 · 2 / 3

Binary file (ikili dosyaya) yazma, güncelleme ve okuma

```
int main() {
    vector<Kayit> v = {yap("Ayse", 101, 84.5), yap("Mehmet", 102, 71),
                       yap("Zeynep", 103, 90), yap("Ali", 104, 65.5),
                       yap("Deniz", 105, 78)};
    const auto B = sizeof(Kayit);
    {
        ofstream y("kayit.dat", ios::binary);
        for (const auto& k : v)
            y.write(reinterpret_cast<const char*>(&k), B);
    }
```

P01 · HAFTA 3 · 3 / 3

Binary file (ikili dosyaya) yazma, güncelleme ve okuma

```
    fstream d("kayit.dat", ios::in | ios::out | ios::binary);
    Kayit k{};
    d.seekg(2 * B);
    d.read(reinterpret_cast<char*>(&k), B);
    cout << "3. kayit: " << k.ad.data() << ' ' << k.puan << '\n';
    k.puan = 95.0;
    d.seekp(2 * B);
    d.write(reinterpret_cast<const char*>(&k), B);
    d.seekg(0, ios::end);
    cout << "boyut: " << d.tellg() << " bayt, " << d.tellg() / B << " kayit\n";
    d.clear();
    d.seekg(0);
    while (d.read(reinterpret_cast<char*>(&k), B))
        cout << k.no << ' ' << k.ad.data() << ' ' << k.puan << '\n';
}
```

ÇIKTI

```
3. kayit: Zeynep 90
boyut: 200 bayt, 5 kayit
101 Ayse 84.5
102 Mehmet 71
103 Zeynep 95
104 Ali 65.5
105 Deniz 78
```

PO2 · HAFTA 3 · 1 / 3

Dosya üzerinde sıralı ve ikili arama

```
#include <fstream>
#include <iostream>
using namespace std;

struct Kayit {
    int no;
    double puan;
};

bool sirali_ara(fstream& d, int anahtar, Kayit& k, int& adim) {
    d.clear();
    d.seekg(0);
    adim = 0;
    while (d.read(reinterpret_cast<char*>(&k), sizeof k)) {
        ++adim;
        if (k.no == anahtar) return true;
    }
    return false;
}
```

PO2 · HAFTA 3 · 2 / 3

Dosya üzerinde sıralı ve ikili arama

```
bool ikili_ara(fstream& d, int anahtar, long n, Kayit& k, int& adim) {
    d.clear();
    long alt = 0, ust = n - 1;
    adim = 0;
    while (alt <= ust) {
        long orta = alt + (ust - alt) / 2;
        d.seekg(orta * sizeof(Kayit));
        d.read(reinterpret_cast<char*>(&k), sizeof k);
        ++adim;
        if (k.no == anahtar) return true;
        if (k.no < anahtar) alt = orta + 1;
        else ust = orta - 1;
    }
    return false;
}

int main() {
    {
        ofstream y("sirali.dat", ios::binary);
        for (int i = 1; i <= 15; ++i) {
            Kayit k{i * 10, 50.0 + i};
            y.write(reinterpret_cast<const char*>(&k), sizeof k);
        }
    }
    fstream d("sirali.dat", ios::in | ios::binary);
    Kayit k{};
    int adim;
```

P02 · HAFTA 3 · 3 / 3

Dosya üzerinde sıralı ve ikili arama

```
    for (int anahtar : {130, 75}) {
        bool s = sirali_ara(d, anahtar, k, adim);
        cout << anahtar << " sirali: " << (s ? "var" : "yok") << ", " << adim << " okuma\n";
        bool b = ikili_ara(d, anahtar, 15, k, adim);
        cout << anahtar << " ikili : " << (b ? "var" : "yok") << ", " << adim << " okuma\n";
    }
}
```

ÇIKTI

```
130 sirali: var, 13 okuma
130 ikili : var, 4 okuma
75 sirali: yok, 15 okuma
75 ikili : yok, 4 okuma
```

P04 · HAFTA 3 · 1 / 3

Öne taşıma, konum değiştirme ve sayaç

```cpp
#include <iostream>
#include <utility>
#include <vector>
using namespace std;

int one_tasi(vector<int>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i] != anahtar) continue;
        for (size_t j = i; j > 0; --j) v[j] = v[j - 1];
        v[0] = anahtar;
        return static_cast<int>(i) + 1;
    }
    return -1;
}

int konum_degistir(vector<int>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i] != anahtar) continue;
        if (i > 0) swap(v[i - 1], v[i]);
        return static_cast<int>(i) + 1;
    }
    return -1;
}
```

P04 · HAFTA 3 · 2 / 3

Öne taşıma, konum değiştirme ve sayaç

```
int sayac_ara(vector<pair<int, int>>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i].first != anahtar) continue;
        ++v[i].second;
        for (size_t j = i; j > 0 && v[j - 1].second < v[j].second; --j)
            swap(v[j - 1], v[j]);
        return static_cast<int>(i) + 1;
    }
    return -1;
}

int main() {
    vector<int> a = {34, 21, 89, 45, 12, 90, 76, 62}, b = a;
    vector<pair<int, int>> c;
    for (int x : a) c.push_back({x, 0});
    int ta = 0, tb = 0, tc = 0;
    for (int x : {90, 89, 90, 21, 90, 90}) {
        ta += one_tasi(a, x);
        tb += konum_degistir(b, x);
        tc += sayac_ara(c, x);
    }
    cout << "one tasima : ";
    for (int x : a) cout << x << ' ';
    cout << "| " << ta << " adim\n";
    cout << "konum degis: ";
    for (int x : b) cout << x << ' ';
    cout << "| " << tb << " adim\n";
```

P04 · HAFTA 3 · 3 / 3

Öne taşıma, konum değiştirme ve sayaç

```
    cout << "sayac      : ";
    for (auto& [x, s] : c) cout << x << ' ';
    cout << "| " << tc << " adim\n";
}
```

ÇIKTI

```
one tasima : 90 21 89 34 45 12 76 62 | 19 adim
konum degis: 34 90 21 89 45 12 76 62 | 24 adim
sayac      : 90 89 21 34 45 12 76 62 | 17 adim
```

PO5 · HAFTA 4 · 1 / 2

Division method (bölme yöntemi) ve progressive overflow

```cpp
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;

int carpisma_say(const vector<int>& anahtarlar, int n) {
    vector<int> sayac(n, 0);
    int carpisma = 0;
    for (int k : anahtarlar)
        if (sayac[k % n]++ > 0) ++carpisma;
    return carpisma;
}

int ekle(vector<int>& tablo, int anahtar, int& probe) {
    const int n = static_cast<int>(tablo.size());
    const int ev = anahtar % n;
    int adres = ev;
    probe = 1;
    while (tablo[adres] != BOS) {
        adres = (adres + 1) % n;
        ++probe;
        if (adres == ev) return -1;
    }
    tablo[adres] = anahtar;
    return adres;
}
```

P05 · HAFTA 4 · 2 / 2

Division method (bölme yöntemi) ve progressive overflow

```
int main() {
    vector<int> h = {71, 23, 73, 99, 44, 49, 60, 20, 33};
    for (int n : {11, 13, 17, 23})
        cout << "mod " << n << ": " << carpisma_say(h, n) << " carpisma\n";
    vector<int> tablo(10, BOS);
    int toplam = 0, probe;
    for (int k : {35, 43, 70, 63, 30, 14, 23, 24, 34}) {
        int a = ekle(tablo, k, probe);
        toplam += probe;
        cout << k << "->" << a << " (" << probe << ")  ";
    }
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0 << '\n';
}
```

ÇIKTI

```
mod 11: 4 carpisma
mod 13: 4 carpisma
mod 17: 1 carpisma
mod 23: 0 carpisma
35->5 (1)  43->3 (1)  70->0 (1)  63->4 (2)  30->1 (2)  14->6 (3)  23->7 (5)  24->8 (5)
34->9 (6)
toplam 26, ortalama 2.88889
```

P07 · HAFTA 7 · 1 / 3

Logical deletion (mantıksal silme)

```cpp
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1, SILINDI = -2;

int ekle(vector<int>& t, int k) {
    const int m = t.size();
    for (int i = 0; i < m; ++i) {
        int a = (k % m + i) % m;
        if (t[a] == BOS || t[a] == SILINDI) { t[a] = k; return a; }
    }
    return -1;
}

int ara(const vector<int>& t, int k, int& probe) {
    const int m = t.size();
    for (probe = 1; probe <= m; ++probe) {
        int a = (k % m + probe - 1) % m;
        if (t[a] == BOS) return -1;
        if (t[a] == k) return a;
    }
    return -1;
}
```

P07 · HAFTA 7 · 2 / 3

Logical deletion (mantıksal silme)

```
bool sil(vector<int>& t, int k) {
    int probe, a = ara(t, k, probe);
    if (a < 0) return false;
    t[a] = SILINDI;
    return true;
}
```

P07 · HAFTA 7 · 3 / 3

Logical deletion (mantıksal silme)

```
int main() {
    vector<int> t(10, BOS);
    for (int k : {35, 43, 70, 63, 30, 14, 23, 24, 34}) ekle(t, k);
    vector<int> yanlis = t;
    int p, a = ara(yanlis, 14, p);
    yanlis[a] = BOS;
    sil(t, 14);
    for (int k : {23, 24, 34}) {
        int p1, p2;
        int a1 = ara(yanlis, k, p1), a2 = ara(t, k, p2);
        cout << k << "  BOS ile: " << a1 << " (" << p1 << ")"
             << "   SILINDI ile: " << a2 << " (" << p2 << ")\n";
    }
    cout << "56 eklendi: " << ekle(t, 56) << '\n';
}
```

ÇIKTI

```
23  BOS ile: -1 (4)   SILINDI ile: 7 (5)
24  BOS ile: -1 (3)   SILINDI ile: 8 (5)
34  BOS ile: -1 (3)   SILINDI ile: 9 (6)
56 eklendi: 6
```

2

Boşluklu kodlar

Sınava hazırlık: önce doldur, sonra çözümle karşılaştır

P08 · SORU · 1 / 3

Coalesced hashing: LISCH ve EISCH

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Goz { int deger = BOS; int baglanti = -1; };

int ekle(vector<Goz>& t, int anahtar, bool erken) {
    const int n = static_cast<int>(t.size());
    const [1] ______ ev = anahtar % n;
    if (t[ev].deger == BOS) { t[ev].deger = anahtar; return ev; }
    int bos = n - 1;
    while (bos >= 0 && t[bos].deger != BOS) --bos;
    if (bos < 0) return -1;
    t[bos].deger = anahtar;
    [2] ______ (erken) {
        t[bos].baglanti = t[ev].baglanti;
        t[ev].baglanti = bos;
    } else {
        int son = ev;
        while (t[son].baglanti != -1) son = t[son].baglanti;
        t[son].baglanti = bos;
    }
    return bos;
}
```

P08 · SORU · 2 / 3

Coalesced hashing: LISCH ve EISCH

```
[3] ______ probe(const vector<Goz>& t, int k) {
    int a = k % int(t.size()), p = 1;
    [4] ______ (t[a].deger != k) { a = t[a].baglanti; ++p; }
    [5] ______ p;
}

int main() {
    const vector<int> anahtarlar = {27, 18, 29, 28, 39, 13, 16, 42, 17};
```

P08 · SORU · 3 / 3

Coalesced hashing: LISCH ve EISCH

```
    [6] ______ ([7] ______ erken : {false, true}) {
        vector<Goz> t(11);
        for (int k : anahtarlar) ekle(t, k, erken);
        int toplam = 0;
        for (int k : anahtarlar) toplam += probe(t, k);
        [8] ______ << (erken ? "EISCH" : "LISCH") << ": toplam " << toplam
             << ", 17 icin " << probe(t, 17) << '\n';
        for (int ev : {5, 6, 7}) {
            cout << "  " << ev << ":";
            for (int a = ev; a != -1; a = t[a].baglanti) cout << ' ' << t[a].deger;
            cout << '\n';
        }
    }
}
```

ÇIKTI

```
LISCH: toplam 16, 17 icin 4
  5: 27 16
  6: 28 39 42 17
  7: 18 29
EISCH: toplam 15, 17 icin 2
  5: 27 16
  6: 28 17 39 42
  7: 18 29
```

P08 · ÇÖZÜM · 1 / 3

Coalesced hashing: LISCH ve EISCH

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Goz { int deger = BOS; int baglanti = -1; };

int ekle(vector<Goz>& t, int anahtar, bool erken) {
    const int n = static_cast<int>(t.size());
    const int ev = anahtar % n;
    if (t[ev].deger == BOS) { t[ev].deger = anahtar; return ev; }
    int bos = n - 1;
    while (bos >= 0 && t[bos].deger != BOS) --bos;
    if (bos < 0) return -1;
    t[bos].deger = anahtar;
    if (erken) {
        t[bos].baglanti = t[ev].baglanti;
        t[ev].baglanti = bos;
    } else {
        int son = ev;
        while (t[son].baglanti != -1) son = t[son].baglanti;
        t[son].baglanti = bos;
    }
    return bos;
}
```

P08 · ÇÖZÜM · 2 / 3

Coalesced hashing: LISCH ve EISCH

```
int probe(const vector<Goz>& t, int k) {
    int a = k % int(t.size()), p = 1;
    while (t[a].deger != k) { a = t[a].baglanti; ++p; }
    return p;
}

int main() {
    const vector<int> anahtarlar = {27, 18, 29, 28, 39, 13, 16, 42, 17};
```

P08 · ÇÖZÜM · 3 / 3

Coalesced hashing: LISCH ve EISCH

```
    for (bool erken : {false, true}) {
        vector<Goz> t(11);
        for (int k : anahtarlar) ekle(t, k, erken);
        int toplam = 0;
        for (int k : anahtarlar) toplam += probe(t, k);
        cout << (erken ? "EISCH" : "LISCH") << ": toplam " << toplam
             << ", 17 icin " << probe(t, 17) << '\n';
        for (int ev : {5, 6, 7}) {
            cout << "  " << ev << ":";
            for (int a = ev; a != -1; a = t[a].baglanti) cout << ' ' << t[a].deger;
            cout << '\n';
        }
    }
}
```

ÇIKTI

```
LISCH: toplam 16, 17 icin 4
  5: 27 16
  6: 28 39 42 17
  7: 18 29
EISCH: toplam 15, 17 icin 2
  5: 27 16
  6: 28 17 39 42
  7: 18 29
```

P09 · SORU · 1 / 2

Brent's metodu

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;

[1] ______ h2(int k) { [2] ______ k / 11; }

[3] ______ brent_ekle(vector<int>& t, int k) {
    const int m = t.size(), ev = k % m, adim = h2(k);
    if (t[ev] == BOS) { t[ev] = k; return; }
    int s = 2, yer = (ev + adim) % m;
    [4] ______ (t[yer] != BOS) { yer = (yer + adim) % m; ++s; }
    [5] ______ (int top = 2; top < s; ++top)
        for (int i = 1; i < top; ++i) {
            int j = top - i;
            int p = (ev + (i - 1) * adim) % m;
            int hedef = (p + j * h2(t[p])) % m;
            [6] ______ (t[hedef] == BOS) {
                t[hedef] = t[p];
                t[p] = k;
                return;
            }
        }
    t[yer] = k;
}
```

P09 · SORU · 2 / 2

Brent's metodu

```
int main() {
    vector<int> t(11, BOS);
    for (int k : {27, 18, 29, 28, 39, 13, 16, 42, 17}) brent_ekle(t, k);
    int toplam = 0;
    for (int a = 0; a < 11; ++a) {
        if (t[a] == BOS) [7] ______;
        int k = t[a], p = k % 11, n = 1;
        while (p != a) { p = (p + h2(k)) % 11; ++n; }
        toplam += n;
        [8] ______ << a << ':' << k << '(' << n << ")  ";
    }
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0 << '\n';
}
```

ÇIKTI

```
0:27(4)  1:42(2)  2:13(1)  4:39(4)  5:16(1)  6:17(1)  7:18(1)  8:28(2)  9:29(2)
toplam 18, ortalama 2
```

P09 · ÇÖZÜM · 1 / 2

Brent's metodu

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;

int h2(int k) { return k / 11; }

void brent_ekle(vector<int>& t, int k) {
    const int m = t.size(), ev = k % m, adim = h2(k);
    if (t[ev] == BOS) { t[ev] = k; return; }
    int s = 2, yer = (ev + adim) % m;
    while (t[yer] != BOS) { yer = (yer + adim) % m; ++s; }
    for (int top = 2; top < s; ++top)
        for (int i = 1; i < top; ++i) {
            int j = top - i;
            int p = (ev + (i - 1) * adim) % m;
            int hedef = (p + j * h2(t[p])) % m;
            if (t[hedef] == BOS) {
                t[hedef] = t[p];
                t[p] = k;
                return;
            }
        }
    t[yer] = k;
}
```

P09 · ÇÖZÜM · 2 / 2

Brent's metodu

```
int main() {
    vector<int> t(11, BOS);
    for (int k : {27, 18, 29, 28, 39, 13, 16, 42, 17}) brent_ekle(t, k);
    int toplam = 0;
    for (int a = 0; a < 11; ++a) {
        if (t[a] == BOS) continue;
        int k = t[a], p = k % 11, n = 1;
        while (p != a) { p = (p + h2(k)) % 11; ++n; }
        toplam += n;
        cout << a << ':' << k << '(' << n << ")  ";
    }
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0 << '\n';
}
```

ÇIKTI

```
0:27(4)  1:42(2)  2:13(1)  4:39(4)  5:16(1)  6:17(1)  7:18(1)  8:28(2)  9:29(2)
toplam 18, ortalama 2
```

P10 · SORU · 1 / 2

Binary tree method (ikili ağaç yöntemi)

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Dugum { int adres, anahtar, ebeveyn; [1] ______ tasima; };

int h2(int k) { return k / 11; }

bool agac_ekle(vector<int>& t, int k) {
    const int m = t.size();
    vector<Dugum> d{{k % m, k, -1, false}};
    [2] ______ (size_t i = 0; i < d.size() && d.size() < 4u * m; ++i) {
        Dugum x = d[i];
        [3] ______ (t[x.adres] == BOS) {
            t[x.adres] = x.anahtar;
            for (int j = i; d[j].ebeveyn != -1; j = d[j].ebeveyn)
                if (d[j].tasima) {
                    const Dugum& e = d[d[j].ebeveyn];
                    t[e.adres] = e.anahtar;
                }
            [4] ______ true;
        }
        d.push_back({(x.adres + h2(x.anahtar)) % m, x.anahtar, int(i), false});
        d.push_back({(x.adres + h2(t[x.adres])) % m, t[x.adres], int(i), true});
    }
    return false;
```

P10 · SORU · 2 / 2

Binary tree method (ikili ağaç yöntemi)

```
}

int main() {
    vector<int> t(11, BOS);
    for (int k : {27, 18, 29, 28, 39, 13, 16, 41, 17, 19}) agac_ekle(t, k);
    [5] ______ toplam = 0;
    for (int a = 0; a < 11; ++a) {
        if (t[a] == BOS) continue;
        int k = t[a], p = k % 11, n = 1;
        [6] ______ (p != a) { p = (p + h2(k)) % 11; ++n; }
        toplam += n;
        [7] ______ << a << ':' << k << '(' << n << ")  ";
    }
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 10.0 << '\n';
}
```

ÇIKTI

```
0:29(3)  1:39(3)  2:13(1)  3:41(3)  5:27(1)  6:16(2)  7:18(1)  8:17(3)  9:19(2)  10:28(3)
toplam 22, ortalama 2.2
```

P10 · ÇÖZÜM · 1 / 2

Binary tree method (ikili ağaç yöntemi)

```cpp
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Dugum { int adres, anahtar, ebeveyn; bool tasima; };

int h2(int k) { return k / 11; }

bool agac_ekle(vector<int>& t, int k) {
    const int m = t.size();
    vector<Dugum> d{{k % m, k, -1, false}};
    for (size_t i = 0; i < d.size() && d.size() < 4u * m; ++i) {
        Dugum x = d[i];
        if (t[x.adres] == BOS) {
            t[x.adres] = x.anahtar;
            for (int j = i; d[j].ebeveyn != -1; j = d[j].ebeveyn)
                if (d[j].tasima) {
                    const Dugum& e = d[d[j].ebeveyn];
                    t[e.adres] = e.anahtar;
                }
            return true;
        }
        d.push_back({(x.adres + h2(x.anahtar)) % m, x.anahtar, int(i), false});
        d.push_back({(x.adres + h2(t[x.adres])) % m, t[x.adres], int(i), true});
    }
    return false;
```

P10 · ÇÖZÜM · 2 / 2

Binary tree method (ikili ağaç yöntemi)

```
}

int main() {
    vector<int> t(11, BOS);
    for (int k : {27, 18, 29, 28, 39, 13, 16, 41, 17, 19}) agac_ekle(t, k);
    int toplam = 0;
    for (int a = 0; a < 11; ++a) {
        if (t[a] == BOS) continue;
        int k = t[a], p = k % 11, n = 1;
        while (p != a) { p = (p + h2(k)) % 11; ++n; }
        toplam += n;
        cout << a << ':' << k << '(' << n << ")  ";
    }
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 10.0 << '\n';
}
```

ÇIKTI

```
0:29(3)  1:39(3)  2:13(1)  3:41(3)  5:27(1)  6:16(2)  7:18(1)  8:17(3)  9:19(2)  10:28(3)
toplam 22, ortalama 2.2
```

P11 · SORU · 1 / 4

Computed chaining

```
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Goz { [1] ______ anahtar = BOS; int carpan = 0; };

int f(int k, int m) { int a = k / m; [2] ______ a == 0 ? 1 : a; }

int sonraki(const vector<Goz>& t, int a) {
    const int m = t.size();
    return (a + t[a].carpan * f(t[a].anahtar, m)) % m;
}

[3] ______ sona_ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    int son = k % m;
    [4] ______ (t[son].carpan != 0) son = sonraki(t, son);
    [5] ______ (int j = 1; j < m; ++j) {
        int a = (son + j * f(t[son].anahtar, m)) % m;
        if (t[a].anahtar == BOS) {
            t[son].carpan = j;
            t[a] = {k, 0};
            return;
        }
    }
}
```

P11 · SORU · 2 / 4

Computed chaining

```
void ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    const int ev = k % m;
    if (t[ev].anahtar == BOS) { t[ev] = {k, 0}; return; }
    const int sahip = t[ev].anahtar % m;
    if (sahip == ev) { sona_ekle(t, k); return; }
    int once = sahip;
    while (sonraki(t, once) != ev) once = sonraki(t, once);
    vector<int> tasinan;
    for (int a = ev;;) {
        tasinan.push_back(t[a].anahtar);
        int s = t[a].carpan ? sonraki(t, a) : -1;
        t[a] = Goz{};
        if (s < 0) [6] ______;
        a = s;
    }
    t[once].carpan = 0;
    t[ev] = {k, 0};
    for (int x : tasinan) sona_ekle(t, x);
}
```

P11 · SORU · 3 / 4

Computed chaining

```
int ara(const vector<Goz>& t, int k) {
    const int m = t.size();
    int a = k % m, probe = 1;
    if (t[a].anahtar == BOS || t[a].anahtar % m != a) return t[a].anahtar == k ? 1 : -1;
    while (t[a].anahtar != k) {
        [7] ______ (t[a].carpan == 0) return -1;
        a = sonraki(t, a);
        ++probe;
    }
    return probe;
}
```

P11 · SORU · 4 / 4

Computed chaining

```
int main() {
    vector<Goz> t(11);
    const vector<int> anahtarlar = {27, 18, 29, 28, 39, 13, 16, 38, 53};
    for (int k : anahtarlar) ekle(t, k);
    for (int a = 0; a < 11; ++a)
        cout << a << ":" << t[a].anahtar << "/" << t[a].carpan << (a % 4 == 3 ? "\n" : "   ");
    int toplam = 0;
    for (int k : anahtarlar) toplam += ara(t, k);
    [8] ______ << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0 << '\n';
}
```

ÇIKTI

```
0:16/1   1:38/0   2:13/0   3:-1/0
4:-1/0   5:27/3   6:28/2   7:18/1
8:29/0   9:53/0   10:39/0
toplam 14, ortalama 1.55556
```

P11 · ÇÖZÜM · 1 / 4

Computed chaining

```cpp
#include <iostream>
#include <vector>
using namespace std;

constexpr int BOS = -1;
struct Goz { int anahtar = BOS; int carpan = 0; };

int f(int k, int m) { int a = k / m; return a == 0 ? 1 : a; }

int sonraki(const vector<Goz>& t, int a) {
    const int m = t.size();
    return (a + t[a].carpan * f(t[a].anahtar, m)) % m;
}

void sona_ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    int son = k % m;
    while (t[son].carpan != 0) son = sonraki(t, son);
    for (int j = 1; j < m; ++j) {
        int a = (son + j * f(t[son].anahtar, m)) % m;
        if (t[a].anahtar == BOS) {
            t[son].carpan = j;
            t[a] = {k, 0};
            return;
        }
    }
}
```

P11 · ÇÖZÜM · 2 / 4

Computed chaining

```
void ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    const int ev = k % m;
    if (t[ev].anahtar == BOS) { t[ev] = {k, 0}; return; }
    const int sahip = t[ev].anahtar % m;
    if (sahip == ev) { sona_ekle(t, k); return; }
    int once = sahip;
    while (sonraki(t, once) != ev) once = sonraki(t, once);
    vector<int> tasinan;
    for (int a = ev;;) {
        tasinan.push_back(t[a].anahtar);
        int s = t[a].carpan ? sonraki(t, a) : -1;
        t[a] = Goz{};
        if (s < 0) break;
        a = s;
    }
    t[once].carpan = 0;
    t[ev] = {k, 0};
    for (int x : tasinan) sona_ekle(t, x);
}
```

P11 · ÇÖZÜM · 3 / 4

Computed chaining

```
int ara(const vector<Goz>& t, int k) {
    const int m = t.size();
    int a = k % m, probe = 1;
    if (t[a].anahtar == BOS || t[a].anahtar % m != a) return t[a].anahtar == k ? 1 : -1;
    while (t[a].anahtar != k) {
        if (t[a].carpan == 0) return -1;
        a = sonraki(t, a);
        ++probe;
    }
    return probe;
}
```

P11 · ÇÖZÜM · 4 / 4

Computed chaining

```
int main() {
    vector<Goz> t(11);
    const vector<int> anahtarlar = {27, 18, 29, 28, 39, 13, 16, 38, 53};
    for (int k : anahtarlar) ekle(t, k);
    for (int a = 0; a < 11; ++a)
        cout << a << ":" << t[a].anahtar << "/" << t[a].carpan << (a % 4 == 3 ? "\n" : "   ");
    int toplam = 0;
    for (int k : anahtarlar) toplam += ara(t, k);
    cout << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0 << '\n';
}
```

ÇIKTI

```
0:16/1   1:38/0   2:13/0   3:-1/0
4:-1/0   5:27/3   6:28/2   7:18/1
8:29/0   9:53/0   10:39/0
toplam 14, ortalama 1.55556
```